Discrete Flow Matching: Tutorial and Code

May 28, 2025

Introduction

This tutorial bridges the gap between the rigorous mathematical formulation of *Discrete Flow Matching* for continuous time Markov chains and a minimal, practical PyTorch implementation. We begin by defining a factorized mixture path between a simple source distribution and an observed target distribution, derive the coordinatewise CTMC velocities via the Kolmogorov forward equation, and show how to assemble those into a learnable model. Each step of the derivation is followed by the code that implements it. You will find:

- A concise statement of each mathematical object (probability path, Bregman divergence, CTMC velocity, posterior parameterization).
- The exact lines of codeno hidden helpersshowing how to (1) sample the mixture path, (2) compute the matching loss, and (3) run the coordinatewise Euler sampler.
- Tips on how to factorize over dimensions so that both training and sampling remain tractable even in high dimensions.

The basic idea is straightforward:

1. Sample a data point
$$X_1 \sim q_{\text{data}}.$$
This is your target (X_i^1) in every coordinate.
2. Sample a source
$$X_0$$
from your simple prior (e.g. uniform or noise).
3. Pick a time
$$t \sim U[0, 1].$$

4. Draw the intermediate state
$$X_t$$
from the mixture path
$$p_{t|0,1}(x \mid X_0, X_1) = \prod_{i=1}^{D} \Big[\kappa(t)\, \delta(x_i,\, X_i^1) + \big(1 - \kappa(t)\big)\, \delta(x_i,\, X_i^0) \Big].$$
In other words, each coordinate i of X_t is X_i^1 with probability $\kappa(t)$, or X_i^0 otherwise.

5. Compute the loss using the known true generator (which depends on X_i^1) and your models posterior
$$p_{1|t}^{\theta,i}(\,\cdot \mid X_t).$$
Because X_i^1 is always the data, it drives the true velocity in every coordinate, and X_t is exactly where you evaluate both true and model velocities.

During inference, you reverse this:

1. Start from X_0 drawn from the prior.
2. Iteratively step forward in t using the learned posterior $p_{1|t}^{\theta,i}$ to sample each coordinates jump, until you reach X_1your generated sample.

Note: X_i^1 in the formulas is always the datas coordinate i. X_t during training is sampled via the chosen mixture path $p_{t|0,1}$, ensuring the model learns to match that paths velocity field.

1 Factorized SourceSink Velocities

Because both the probability path and its velocity factorize over coordinates, we may treat each dimension independently. Writing a full state as $x = (x_1, \ldots, x_d)$ and a proposed next state as $y = (y_1, \ldots, y_d)$, we have
$$q_t(x) = \prod_{i=1}^{d} q_t^i(x_i),$$
and the full rate vector
$$u_t(y, x) = \sum_{i=1}^{d} \frac{\underbrace{\delta\big(y_{\bar{i}},\, x_{\bar{i}}\big)}}{\prod_{j \neq i} \delta(y_j, x_j)} \quad u_t^i\big(y_i,\, x\big).$$
Here
$$\delta(y_{\bar{i}}, x_{\bar{i}}) = \prod_{j \neq i} \delta(y_j, x_j)$$

ensures that only coordinate i is allowed to move, and

$$u_t^i(y_i, x) \in \mathbb{R}$$

is exactly the *sourcesink rate* for changing coordinate i from its current value x_i (the source) to a new value y_i (the sink). By factorizing both q_t and u_t this way, we reduce the high-dimensional CTMC rate problem into d independent scalar rate functions u_t^i.

2 Mixture-path Construction

Introduce an auxiliary coupling $Z = (X_0, X_1)$. For each coordinate $i \in \{1, \dots, d\}$, define the conditional marginal path

$$p_{t|0,1}^i(x_i \mid x_0, x_1) \;=\; \kappa(t)\,\delta\big(x_i, x_{1,i}\big) \;+\; \big(1 - \kappa(t)\big)\,\delta\big(x_i, x_{0,i}\big)\,,$$

where we choose the simple scheduler

$$\kappa(t) = t, \qquad \dot{\kappa}(t) = 1.$$

Under this choice, at time t each coordinate X_t^i equals its sink value X_1^i with probability t, and its source value X_0^i with probability $1 - t$. In other words,

$$\Pr\big[X_t^i = X_1^i \mid X_0 = x_0,\, X_1 = x_1\big] = t, \qquad \Pr\big[X_t^i = X_0^i \mid X_0 = x_0,\, X_1 = x_1\big] = 1{-}t.$$

Thus the full factorized path is

$$p_{t|0,1}(x \mid x_0, x_1) \;=\; \prod_{i=1}^{d} p_{t|0,1}^i(x_i \mid x_0, x_1)\,.$$

3 Posteriordriven SourceSink Velocities

Once we have the pure sourcesink rates

$$u_t^i(y_i, x \mid x_0, x_1) = \frac{1}{1-t}\Big[\delta(y_i, x_{1,i}) - \delta(y_i, x_i)\Big]$$

for the mixture path, we introduce a learnable percoordinate targetinitial posterior

$$p_{\theta,i,1|t}(x_{1,i} \mid x) \approx p_{i,1|t}(x_{1,i} \mid x)\,.$$

We then assemble the marginal velocity as

$$u_t^i(y_i, x) = \sum_{x_{1,i}} u_t^i(y_i, x \mid x_0, x_1)\; p_{\theta,i,1|t}(x_{1,i} \mid x) = \frac{1}{1-t} \sum_{x_{1,i}} \Big[\delta(y_i, x_{1,i}) {-} \delta(y_i, x_i)\Big]\; p_{\theta,i,1|t}(x_{1,i} \mid x).$$

Plugging this into the Eulerstep transition kernel

$$P\big(X_{t+h,i} = y_i \mid X_t = x\big) = \delta(y_i, x_i) + h\, u_t^i(y_i, x) + o(h)$$

yields

$$P\big(X_{t+h,i} = y_i \mid X_t = x\big) = \sum_{x_{1,i}} \Big[\delta(y_i, x_i) + \tfrac{h}{1-t}\big(\delta(y_i, x_{1,i}) - \delta(y_i, x_i)\big)\Big]\, p_{\theta,i,1|t}(x_{1,i} \mid x) + o(h).$$

In practice, sampling is implemented coordinatewise by:

1. Draw $x_{1,i} \sim p_{\theta,i,1|t}(\cdot \mid x)$.
2. With probability $\frac{h}{1-t}$, set $X_{t+h,i} = x_{1,i}$; otherwise keep $X_{t+h,i} = x_i$.

This makes explicit how the learned posterior per coordinate drives the sourcesink transitions during sampling.

4 Coordinatewise Euler Sampling

For a small time step h, the CTMC transition kernel factorizes over coordinates:

$$P\big(X_{t+h,i} = y_i \mid X_t = x\big) = \delta(y_i, x_i) + h\, u_t^i(y_i, x) + o(h).$$

Substituting our posteriorassembled velocity

$$u_t^i(y_i, x) = \frac{1}{1-t} \sum_{x_{1,i}} \big[\delta(y_i, x_{1,i}) - \delta(y_i, x_i)\big]\, p_{\theta,i,1|t}(x_{1,i} \mid x),$$

we obtain

$$P\big(X_{t+h,i} = y_i \mid X_t = x\big) = \sum_{x_{1,i}} \Big[\delta(y_i, x_i) + \frac{h}{1-t}\big(\delta(y_i, x_{1,i}) - \delta(y_i, x_i)\big)\Big]\, p_{\theta,i,1|t}(x_{1,i} \mid x) + o(h).$$

Hence the coordinatewise sampling algorithm is:

1. Draw $x_{1,i} \sim p_{\theta,i,1|t}(\cdot \mid x)$.
2. With probability $\frac{h}{1-t}$, set $X_{t+h,i} = x_{1,i}$; otherwise keep $X_{t+h,i} = x_i$.

5 PerCoordinate Posterior Parameterization and Training Loss

We parameterize the marginal targetinitial posterior per coordinate

$$p_{\theta,i,1|t}(x_{1,i} \mid x_t)$$

via a neural network (embedded and flattened as in Section **??**). By Proposition 1 the Conditional Discrete Flow Matching loss over our mixture path reduces to a sum of generalizedKL Bregman divergences per coordinate:

$$L(\theta) = \mathbb{E}_{t, X_0, X_1, X_t}\left[\sum_{i=1}^{d} D_{\mathrm{gKL}}\Big(u_t^i(\cdot, x_t \mid x_0, x_1)\,,\; u_t^{\theta,i}(\cdot, x_t)\Big)\right].$$

For the mixturepath generator

$$u_t^i(y_i, x_t \mid x_0, x_1) = \frac{1}{1-t}\big[\delta(y_i, x_{1,i}) - \delta(y_i, x_{t,i})\big],$$

choosing D_{gKL} yields the simple crossentropy objective

$$L(\theta) = -\mathbb{E}_{t, X_0, X_1, X_t} \sum_{i=1}^{d} \log p_{\theta, i, 1|t}\big(X_{1,i} \mid X_t\big) \; + \; \text{const.}$$

In code, using the librarys generalizedKL loss wrapper:

```
from flow_matching.loss import MixturePathGeneralizedKL

loss_fn = MixturePathGeneralizedKL(path=path)

for x_0, x_1 in dataloader:
t      = torch.rand(batch_size) * (1.0 - 1e-3)
sample = path.sample(t=t, x_0=x_0, x_1=x_1)
logits = model(sample.x_t, sample.t)
loss   = loss_fn(
logits=logits,
x_1=sample.x_1,
x_t=sample.x_t,
t=sample.t
)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```